

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



TREE AND GRAPH

Oleh:

Bi'ahlil Haq Aulia Akbar Awaludin

NIM. 2010817110011

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Tree dan Graph ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Bi'ahlil Haq Aulia Akbar Awaludin
NIM : 2010817110011

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	i
DAFTAR ISI.....	ii
DAFTAR GAMBAR.....	iii
DAFTAR TABEL.....	iv
SOAL 1.....	1
A. Source Code.....	3
B. Output Program.....	4
C. Pembahasan.....	4
SOAL 2.....	1
A. Source Code.....	2
B. Output Program.....	4
C. Pembahasan.....	4
SOAL 3.....	1
A. Source Code.....	3
B. Output Program.....	4
C. Pembahasan.....	5
SOAL 4.....	1
A. Source Code.....	3
B. Output Program.....	4
C. Pembahasan.....	4
SOAL 5.....	7
A. Source Code.....	9
B. Output Program.....	11
C. Pembahasan.....	11
TAUTAN GIT.....	14

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1.....	4
Gambar 2. Screenshot Hasil Jawaban Soal 2.....	4
Gambar 3. Screenshot Hasil Jawaban Soal 3.....	5
Gambar 4. Screenshot Hasil Jawaban Soal 4.....	4
Gambar 5. Screenshot Hasil Jawaban Soal 5.....	11

DAFTAR TABEL

<i>Table 1. Source Code soal1.cpp</i>	4
<i>Table 2. Source Code soal2.cpp</i>	3
<i>Table 3. Source Code soal3.cpp</i>	4
<i>Table 4. Source Code soal4.cpp</i>	4
<i>Table 5. Source Code soal5.cpp</i>	11

SOAL 1

Konversi Adjacency List ke Adjacency Matrix

Deskripsi Soal

Diberikan sebuah directed weighted graph yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U,V,W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U,V,W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi adjacency matrix. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

M

$U_1 V_1 W_1$

$U_2 V_2 W_2$

...

$U_M V_M W_M$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

$V_1 V_2 \dots V_N$

$V_1 a_{11} a_{12} \dots a_{1N}$

$V_2 a_{21} a_{22} \dots a_{2N}$

...

$V_N a_{N1} a_{N2} \dots a_{NN}$

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki self-loop.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E	Adjacency Matrix: A B C D E
6 A B 4 A C 2 B D 7 C B 1	A 0 4 2 0 0 B 0 0 0 7 0 C 0 1 0 0 6 D 0 0 3 0 0 E 0 0 0 0 0

CE 6	
DC 3	

A. Source Code

soal1.cpp

```
1  #include <iostream>
2  #include <map>
3  #include <vector>
4  using namespace std;
5
6  int main() {
7
8      // Vertex / node
9      int N;
10     cin >> N;
11
12     vector<char> label(N);
13     for (int i = 0; i < N; i++) {
14         cin >> label[i];
15     }
16
17     // Buat mapping char
18     map<char, int> adj;
19     for (int i = 0; i < N; i++) {
20         adj[label[i]] = i;
21     }
22
23     // Inisialisasi matrix N x N dengan 0
24     vector<vector<int>> matrix(N, vector<int>(N, 0));
25
26     // Edge / Busur / panah
27     int M;
28     cin >> M;
29
30     // Isi matrix
31     for (int i = 0; i < M; i++) {
32         char u, v;
33         int w;
34         cin >> u >> v >> w;
35         matrix[adj[u]][adj[v]] = w;
36     }
```



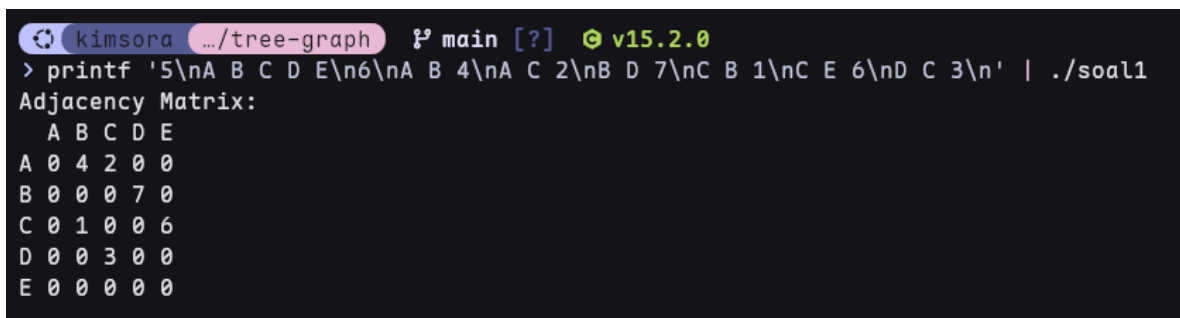
```

37
38 // Cetak text header
39 cout << "Adjacency Matrix:" << endl;
40
41 // Cetak header matrix
42 cout << " ";
43 for (int i = 0; i < N; i++) {
44     if (i > 0)
45         cout << " ";
46     cout << label[i];
47 }
48 cout << endl;
49
50 // Cetak setiap baris matrix
51 for (int i = 0; i < N; i++) {
52     cout << label[i];
53     for (int j = 0; j < N; j++) {
54         cout << " " << matrix[i][j];
55     }
56     cout << endl;
57 }
58
59 return 0;
60 }

```

Table 1. Source Code soal1.cpp

B. Output Program



```

kimsora .../tree-graph  ? main [?]  v15.2.0
> printf '5\nA B C D E\n6\nA B 4\nA C 2\nB D 7\nC B 1\nC E 6\nD C 3\n' | ./soal1
Adjacency Matrix:
 A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0

```

Gambar 1. Screenshot Hasil Jawaban Soal 1

C. Pembahasan

Kompleksitas waktu

Karena disini ada dua variable yang menjadi patokan seberapa banyak perulangan akan terjadi yaitu N (banyak node) dan M (banyak edge / panah) maka kompleksitas waktu nya adalah $O(n m)$.

Kompleksitas ruang

Untuk menampung node diperlukan vector dengan size N, dan ada juga vector N x N (nested vector) yang digunakan sebagai penyimpan matrix, disini perbedaan nya tidak ada ruang yang bergantung pada M. Ini menjadikan kompleksitas ruang nya menjadi $O(n^2)$.

Karakteristik problem

Problem ini meminta untuk mengubah adjacency list menjadi matriks adjacency. Kita perlu menyimpan input banyak nya node dan juga label nya karena itu kita menyimpan banyak nya node dan juga label untuk setiap node dengan vector char. Kemudian kita lakukan mapping label agar memiliki urutan dengan map<char, int>. Untuk memasukan berapa banyak nya edge yang ingin dibuat kita simpan pada variable m, setelah ini kita perlu lakukan loop untuk memasukan list dan sekaligus mengubah nya menjadi nilai di matriks, dengan cara $matrix[adj[u]][adj[v]] = w$.

SOAL 2

Konversi Adjacency Matrix ke Adjacency List

Deskripsi Soal

Diberikan sebuah directed weighted graph yang direpresentasikan dalam bentuk adjacency matrix berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi adjacency list. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

$A_{11} A_{12} \dots A_{1N}$

$A_{21} A_{22} \dots A_{2N}$

...

$A_{N1} A_{N2} \dots A_{NN}$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .
- Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .
- Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency List:

$V_1 \rightarrow (X_1, w_1), (X_2, w_2), \dots$

V2 -> (X1,w1), (X2,w2), ...

...

VN -> ...

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol ->.

Batasan

- $2 \leq N \leq 10$
- $0 \leq A_{ij} \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge.
- Nilai positif berarti bobot edge.
- Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 0 4 2 0 0 0 0 0 7 0 0 1 0 0 6 0 0 3 0 0 0 0 0 0 0	Adjacency List: A -> (B,4), (C,2) B -> (D,7) C -> (B,1), (E,6) D -> (C,3) E -> -

A. Source Code

soal2.cpp

1	#include <iostream>
2	#include <vector>

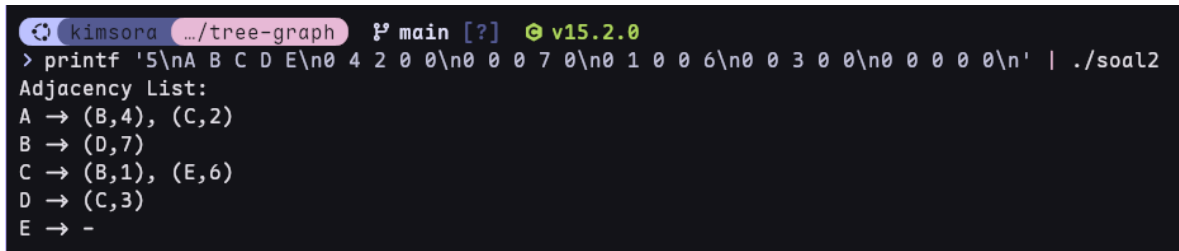
```

3 using namespace std;
4
5 int main() {
6     // Banyak Label
7     int N;
8     cin >> N;
9
10    // Baca Label Karakter
11    vector<char> label(N);
12    for (int i = 0; i < N; i++) {
13        cin >> label[i];
14    }
15
16    // Baca Matrix adjacency
17    vector<vector<int>> matrix(N, vector<int>(N));
18    for (int i = 0; i < N; i++) {
19        for (int j = 0; j < N; j++) {
20            cin >> matrix[i][j];
21        }
22    }
23
24    // Cetak header dan matrix adjacency list
25    cout << "Adjacency List:" << endl;
26    for (int i = 0; i < N; i++) {
27        cout << label[i] << " -> ";
28        bool hasEdge = false;
29        for (int j = 0; j < N; j++) {
30            if (matrix[i][j] > 0) {
31                if (hasEdge) {
32                    cout << ", ";
33                }
34                cout << "(" << label[j] << ", " << matrix[i][j] <<
35                ")\n";
36                hasEdge = true;
37            }
38        }
39        if (!hasEdge) {
40            cout << "-\n";
41        }
42        cout << endl;
43    }
44
45    return 0;
46 }

```

Table 2. Source Code soal2.cpp

B. Output Program



```
kimsora .../tree-graph main [?] v15.2.0
> printf '5\nA B C D E\n0 4 2 0 0\n0 0 0 7 0\n0 1 0 0 6\n0 0 3 0 0\n0 0 0 0 0\n' | ./soal2
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
```

Gambar 2. Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Kompleksitas waktu

Untuk membaca matriks yang diberikan pada input kita perlu melakukan nested loop yang bergantung dengan banyak nya N (node), dan untuk mencetak adjacency list kita juga perlu melakukan nested loop yang bergantung terhadap N juga, jadi ini membuat kompleksitas waktu nya adalah Big-O notation $O(n^2)$.

Kompleksitas ruang

Sama seperti soal 1 kita masih memerlukan nested vector yang kedua nya sebanyak N (matriks $N \times N$) jadi kompleksitas ruang nya sama yaitu Big-O notation $O(n^2)$.

Karakteristik problem

Sama seperti sebelum nya problem ini memerlukan kita untuk membaca label node, yang membedakan nya disini kita perlu menerima input matriks, untuk menerima itu kita perlu nested vector sebagai wadah matriks, dan nested loop untuk mengisi value setiap cell pada matriks dengan cin `>> matrix[i][j]`. Inti dari problem ini adalah mengubah matriks menjadi adjacency list disini lah kita akan transform dari matriks ke list dengan cara kita loop sebanyak N untuk setiap label, baru didalam nya ada loop lagi untuk menentukan apakah node ini ada edge nya diperlukan komparasi `matrix[i][j] > 0` dan cetak edge yang ditemukan.

SOAL 3

BFS: Jarak Terpendek Keluar Labirin

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma Breadth-First Search (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

R C

G_{11} G_{12} ... G_{1C}

G_{21} G_{22} ... G_{2C}

...

G_{R1} G_{R2} ... G_{RC}

S_R S_C

F_R F_C

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak adjacency matrix dengan format berikut.

X

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1 .

Batasan

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Input	Output
8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0	16

0000100010	
0110001000	
00	
79	

A. Source Code

soal3.cpp

```
1  #include <iostream>
2  #include <queue>
3  #include <vector>
4  using namespace std;
5
6  int main() {
7      int R, C;
8      cin >> R >> C;
9
10     // grid R x C
11     vector<vector<int>> grid(R, vector<int>(C));
12     for (int i = 0; i < R; i++) {
13         for (int j = 0; j < C; j++) {
14             cin >> grid[i][j];
15         }
16     }
17
18     // posisi awal dan tujuan
19     int sr, sc, fr, fc;
20     cin >> sr >> sc;
21     cin >> fr >> fc;
22
23     // Inisialisasi visited dan dist
24     vector<vector<bool>> visited(R, vector<bool>(C,
25 false));
26     vector<vector<int>> dist(R, vector<int>(C, -1));
27
28     // Definisikan 4 arah pergerakan
29     int dr[] = {-1, 1, 0, 0};
30     int dc[] = {0, 0, -1, 1};
31
32     // Inisialisasi queue dan push start
33     queue<pair<int, int>> q;
```

```

34     q.push({sr, sc});
35     visited[sr][sc] = true;
36     dist[sr][sc] = 0;
37
38     // BFS Loop
39     while (!q.empty()) {
40         auto [r, c] = q.front();
41         q.pop();
42
43         // Cek apakah sudah sampai tujuan
44         if (r == fr && c == fc) {
45             cout << dist[r][c] << endl;
46             return 0;
47         }
48
49         // Cek 4 arah
50         for (int i = 0; i < 4; i++) {
51             int nr = r + dr[i];
52             int nc = c + dc[i];
53
54             // Validasi batas grid + bukan dinding + belum
55             visited
56                 if (nr >= 0 && nr < R && nc >= 0 && nc < C &&
57                 grid[nr][nc] == 0 &&
58                 !visited[nr][nc]) {
59                 visited[nr][nc] = true;
60                 dist[nr][nc] = dist[r][c] + 1;
61                 q.push({nr, nc});
62             }
63         }
64     }
65     cout << -1 << endl;
66     return 0;
67 }

```

Table 3. Source Code soal3.cpp

B. Output Program



```

kimsora .../tree-graph 1? main [?] v15.2.0
> printf '8 10\n0 0 0 0 1 0 0 0 0\n1 1 0 1 0 1 1 1 0\n0 0 1 0 0 0 0 1 0\n0 1 1 1 1
1 0 1 0\n0 0 0 0 0 1 0 0 0\n1 1 1 1 0 1 1 1 0\n0 0 0 0 1 0 0 1 0\n0 1 1 0 0 0 1 0 0 0
\n0 0\n7 9\n' | ./soal3
16

```

C. Pembahasan

Kompleksitas waktu

Karena kita membaca grid jadi kompleksitas waktu nya adalah $O(r \cdot c)$ sedangkan untuk bfs nya sendiri juga adalah $O(r \cdot c)$ pada worst case karena dia akan melakukan loop semua node.

Kompleksitas ruang

Kompleksitas ruang nya disini adalah $O(r \cdot c)$ karena ada 3 buah vector $R \times C$ yang selalu penuh dan 1 queue yang worst case bisa mencapai $O(r \cdot c)$ juga. Jadi Big-O notation nya $O(r \cdot c)$

Karakteristik problem

Problem ini adalah sebuah labirin yang direpresentasikan oleh grid berukuran $R \times C$. dengan isi 0 jika jalan bisa dilewati dan 1 jika itu dinding yang tidak bisa dilewati, kita diminta untuk mencari jumlah langkah minimal untuk mencapai tujuan, dengan syarat kita tidak bisa bergerak diagonal. Artinya kita diminta menggunakan Breadth-First Search. Dan input nya adalah R, C, Matriks, Koordinat awal, Koordinat tujuan.

Untuk R dan C standart saja input simpan di variable, untuk matriks juga standart input disimpan dengan cara nested loop sesuai index nya. Hal yang penting disini adalah kita perlu membuat grid berukuran sama yang menyimpan nilai true false sebagai penunjuk apakah langkah sudah dilalui. Kita juga membuat satu grid berukuran sama lagi untuk mencatat jarak yang tengah dilalui saat ini. Sebelum kita mulai loop utama bfs kita perlu init queue pair<int, int> dan push sr,sc kedalam nya, set visited[sr][sc] = true dan dist[sr][sc] = 0. Kita juga perlu mendefinisikan 4 arah pergerakan dengan cara membuat 2 array yaitu dr (row) dc (kolom) dimana dr[] = {-1, 1, 0, 0}; yang menandakan jika bergerak ke atas row nya -1, jika kebawah row nya +1, dan dc[] = {0, 0, -1, 1}; yang menandakan jika bergerak kiri kolom nya -1 jika kanan kolom nya +1 (index / kordinat).

Pada loop bfs dengan kondisi while (!q.empty()) ini kita perlu mengambil nilai koordinat dari pair yang ada di front queue dengan cara deconstructor menjadi r dan c, kemudian kita pop / dequeue, karena kita sudah mengambil nilainya dan akan melangkah dari titik ini jadi titik ini tidak perlu disimpan di queue. Kemudian kita lakukan cek apakah koordinat saat ini sudah sama dengan koordinat tujuan jika sudah cetak nilai dari vector dist dengan koordinat saat

ini. Nah disini kita akan melakukan langkah atau cek 4 arah dengan cara loop 4x disini kita simpan kordinat baru nya dengan cara $nr = r + dr[i]$ dan $nc = c + dc[i]$. dc dan dr berfungsi sebagai pengubah kordinat (langkah) yang sudah kita tentukan sebelumnya. Setelah itu dilakukan validasi batas grid + bukan dinding + belum visited, dan jika kordinat saat ini valid baru kita tandai kordinat ini true di visited grid dan juga simpan distace nya di grid dist dengan cara $dist[nr][nc] = dist[r][c] + 1$; baru kita push kordinat ini sebagai pair kedalam queue.

Dan ini akan diulang terus sampai queue empty atau sudah sampai tujuan. Jadi loop akan mengambil nilai pair pada queue depan dan dequeue nya, lalu lakukan apakah nilai dari pair sudah sama dengan tujuan? Jika ya print jarak, jika tidak proses 4 langkah, langkah yang valid akan masuk ke queue.

SOAL 4

DFS: Menghitung Semua Jalur Keluar Labirin

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma Depth-First Search (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses backtracking agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

R C

G_{11} G_{12} ... G_{1C}

G_{21} G_{22} ... G_{2C}

...

G_{R1} G_{R2} ... G_{RC}

S_R S_C

F_R F_C

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak adjacency matrix dengan format berikut.

T

Keterangan:

- T adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak 0.

Batasan

- $1 \leq R \leq 7$
- $1 \leq C \leq 7$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0	4

00	
45	

A. Source Code

soal4.cpp

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  void dfs(int r, int c, int fr, int fc, int R, int C,
6           const vector<vector<int>> &grid,
7           vector<vector<bool>> &visited,
8           int &pathCount) {
9
10     if (r == fr && c == fc) {
11         pathCount++;
12         return;
13     }
14     visited[r][c] = true;
15
16     // Petunjuk 4 arah
17     int dr[] = {-1, 1, 0, 0};
18     int dc[] = {0, 0, -1, 1};
19
20     // Cek 4 arah
21     for (int i = 0; i < 4; i++) {
22         int nr = r + dr[i];
23         int nc = c + dc[i];
24         // Validasi
25         if (nr >= 0 && nr < R && nc >= 0 && nc < C && grid[nr]
26 [nc] == 0 &&
27         !visited[nr][nc]) {
28             dfs(nr, nc, fr, fc, R, C, grid, visited,
29 pathCount);
30         }
31     }
32     // Backtrack
33     visited[r][c] = false;
34 }
35
36 int main() {
```

```

37  int R, C;
38  cin >> R >> C;
39
40  vector<vector<int>> grid(R, vector<int>(C));
41  for (int i = 0; i < R; i++) {
42      for (int j = 0; j < C; j++) {
43          cin >> grid[i][j];
44      }
45  }
46
47  int sr, sc, fr, fc;
48  cin >> sr >> sc;
49  cin >> fr >> fc;
50
51      vector<vector<bool>> visited(R, vector<bool>(C,
52 false));
53  int pathCount = 0;
54  dfs(sr, sc, fr, fc, R, C, grid, visited, pathCount);
55  cout << pathCount << endl;
56
57  return 0;
58 }

```

Table 4. Source Code soal4.cpp

B. Output Program



```

kimsora ~/tree-graph 1? main [?] v15.2.0
> printf '5 6\n0 0 0 1 0 0\n0 1 0 0 0 1\n0 1 0 1 0 0\n0 0 0 1 1 0\n1 1 0 0 0 0\n0 0 0\n4 5\n'
| ./soal4
4

```

Gambar 4. Screenshot Hasil Jawaban Soal 4

C. Pembahasan

Kompleksitas waktu

Karena algoritma yang digunakan adalah dfs + backtracking untuk mencari semua jalur yang ada, berbeda dengan dfs normal yang kompleksitas nya adalah $O(r \cdot c)$ / $O(n^2)$. Karena backtracking ini akan membuat nya kembali satu langkah dan mencoba langkah lain, dan ini membuat kompleksitas nya menjadi eksponensial, untuk menentukan kompleksitas waktu nya kita perlu mengetahui kondisi dari case kita yaitu langkah 4 arah, dan tidak bisa langkah

mundur yang membuat kita mempunyai 3 case setiap langkah. Dan kita perlu melakukan ini untuk semua node pada grid yang menjadikan kompleksitas waktunya adalah $O(3^{R \times C})$. Karena pada worst case nya akan ada 3 case untuk setiap node.

Kompleksitas ruang

Yah masih sama seperti soal sebelumnya memiliki 2 vector grid $R \times c$ yang membuat nya $O(r \times c)$, yang membedakan nya disini adalah adanya recursif pemanggilan yang membuat call stack yang bergantung kepada R dan C . karena kedua nya memiliki kompleksitas $O(r \times c)$

Karakteristik problem

Sama seperti soal 3 kita masih di labirin. Namun disini kita diminta untuk menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma Depth-First Search. Dengan aturan tidak boleh mengunjungi sel yang sama lebih dari 1 kali dan tidak boleh bergerak diagonal cuman 4 arah. Input yang diterima juga sama dengan soal sebelumnya.

Untuk menyelesaikan problem ini, kita memerlukan sebuah array visited berukuran sama dengan grid yang bersifat sementara. Berbeda dengan soal sebelumnya di mana visited bersifat permanen setelah sebuah sel diproses, di sini visited harus bisa dikembalikan setelah selesai menelusuri satu cabang jalur. Prosesnya dimulai dengan memanggil fungsi rekursif dfs dari koordinat awal. Di dalam fungsi ini, pertama kita periksa apakah posisi saat ini sudah sama dengan koordinat tujuan. Jika ya, kita tingkatkan nilai counter jalur sebanyak satu lalu return.

Sebelum melanjutkan ke empat arah pergerakan, koordinat saat ini harus ditandai sebagai visited agar tidak terjadi pengulangan sel dalam satu jalur yang sedang ditelusuri. Kemudian untuk masing-masing dari empat arah yang mungkin, kita hitung koordinat baris dan kolom baru menggunakan pair array dr dan dc yang menentukan . Setiap koordinat baru yang lolos validasi, yaitu berada dalam batas grid, bukan dinding, dan belum dikunjungi pada jalur ini, akan menjadi titik awal pemanggilan rekursif berikutnya.

Ketika pemanggilan rekursif untuk suatu arah selesai dan return, yang terjadi adalah kita telah menyelesaikan eksplorasi seluruh cabang yang dimulai dari sel tersebut. Pada titik inilah mekanisme backtracking berperan penting: kita harus mengubah status visited untuk koordinat saat ini kembali menjadi false. Tindakan ini memberikan kesempatan pada jalur-jalur lain yang mungkin melintasi sel yang sama namun melalui rute berbeda untuk tetap bisa dieksplorasi. Tanpa langkah ini, program hanya akan menemukan satu jalur pertama dan mengabaikan semua kemungkinan alternatif.

Proses rekursi dan backtracking ini terus berlanjut hingga seluruh kemungkinan jalur sederhana telah diperiksa. Counter jalur yang telah ditambah setiap kali mencapai tujuan kemudian dicetak sebagai output akhir. Jika tidak ada satupun jalur yang valid, counter tetap bernilai nol dan itulah yang akan ditampilkan.

SOAL 5

Tree Traversal

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah Red-Black Tree secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTree terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.
- INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.
- POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.
- ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

N

$A_1 A_2 \dots A_N$

Q

T_1

T_2

...

T_Q

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTree.
- A_i adalah bilangan ke- i .
- Q adalah banyaknya query traversal.
- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

[Preorder] : daftar_angka

[Inorder] : daftar_angka

[Postorder] : daftar_angka

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

Batasan

- $0 \leq N \leq 1000$
- $-10000000000 \leq A_i \leq 10000000000$
- $1 \leq Q \leq 20$
- T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Input	Output
7	[Preorder] : 50 30 20 40 70 60 80
50 30 70 20 40 60 80	[Inorder] : 20 30 40 50 60 70 80
4	[Postorder] : 20 40 30 60 80 70 50
PREORDER	[Preorder] : 50 30 20 40 70 60 80
INORDER	[Inorder] : 20 30 40 50 60 70 80
POSTORDER	[Postorder] : 20 40 30 60 80 70 50
ALL	

A. Source Code

soal5.cpp

```
1  #include "RedBlackTree.h"
2  #include <iostream>
3  #include <string>
4  using namespace std;
5
6  void preorderHelper(const RedBlackTree::Node *node,
7                     const RedBlackTree::Node *nil) {
8      if (node == nil || node->isNil)
9          return;
10     cout << node->key << " ";
11     preorderHelper(node->left, nil);
12     preorderHelper(node->right, nil);
13 }
14
15 void inorderHelper(const RedBlackTree::Node *node,
16                   const RedBlackTree::Node *nil) {
17     if (node == nil || node->isNil)
18         return;
19     inorderHelper(node->left, nil);
20     cout << node->key << " ";
21     inorderHelper(node->right, nil);
22 }
23
24 void postorderHelper(const RedBlackTree::Node *node,
25                     const RedBlackTree::Node *nil) {
26     if (node == nil || node->isNil)
27         return;
28     postorderHelper(node->left, nil);
29     postorderHelper(node->right, nil);
30     cout << node->key << " ";
31 }
32
33 void preorder(const RedBlackTree &tree) {
34     preorderHelper(tree.root(), tree.nil());
35 }
36
37 void inorder(const RedBlackTree &tree) {
38     inorderHelper(tree.root(), tree.nil());
39 }
40
41 void postorder(const RedBlackTree &tree) {
42     postorderHelper(tree.root(), tree.nil());
```

```

43 }
44
45
46 int main() {
47     // Banyak nya Node
48     int N;
49     cin >> N;
50
51     RedBlackTree tree;
52
53     // Insert node
54     for (int i = 0; i < N; i++) {
55         int val;
56         cin >> val;
57         if (!tree.contains(val)) {
58             tree.insert(val);
59         }
60     }
61
62     if (tree.empty()) {
63         cout << "Tree kosong. Tidak ada yang bisa
64 ditampilkan." << endl;
65         return 0;
66     }
67
68     // Banyaknya Langkah Travers
69     int Q;
70     cin >> Q;
71
72     for (int i = 0; i < Q; i++) {
73         string query;
74         cin >> query;
75         if (query == "PREORDER") {
76             cout << "[Preorder] : ";
77             preorder(tree);
78             cout << endl;
79         } else if (query == "INORDER") {
80             cout << "[Inorder] : ";
81             inorder(tree);
82             cout << endl;
83         } else if (query == "POSTORDER") {
84             cout << "[Postorder] : ";
85             postorder(tree);
86             cout << endl;

```

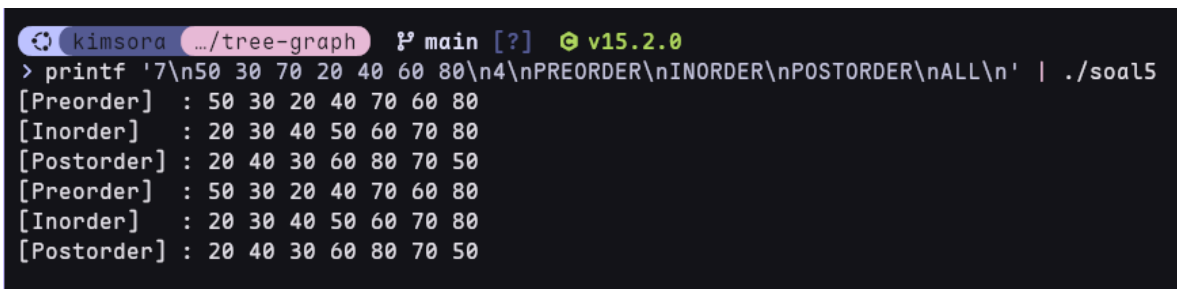
```

87     } else if (query == "ALL") {
88         cout << "[Preorder]  : ";
89         preorder(tree);
90         cout << endl;
91         cout << "[Inorder]   : ";
92         inorder(tree);
93         cout << endl;
94         cout << "[Postorder] : ";
95         postorder(tree);
96         cout << endl;
97     }
98 }
99
100 return 0;
101 }

```

Table 5. Source Code soal5.cpp

B. Output Program



```

kimsora .../tree-graph main [?] v15.2.0
> printf '7\n50 30 70 20 40 60 80\n4\nPREORDER\nINORDER\nPOSTORDER\nALL\n' | ./soal5
[Preorder] : 50 30 20 40 70 60 80
[Inorder]  : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
[Preorder] : 50 30 20 40 70 60 80
[Inorder]  : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50

```

Gambar 5. Screenshot Hasil Jawaban Soal 5

C. Pembahasan

Kompleksitas waktu

Metode insert dan contains yang diberikan pada rbt memiliki kompleksitas $O(\log n)$ dan traversal yang akan selalu mengunjungi setiap node maka kompleksitas nya $O(n)$ namun traversal disini dijalankan di atas loop Q yang menjadikan Kompleksitas gabungannya adalah $O(qn)$.

Kompleksitas ruang

Karena rekursi traversal mendalam maksimal tinggi tree membentuk call stack = $O(\log N)$ karena RBT nya seimbang. Dan node pada tree sendiri sebanyak $O(n)$ maka $O(\log N) + O(n)$ tetap membuat nya menjadi $O(n)$

Karakteristik problem

Problem ini adalah manipulasi struktur data binary tree yang self-balancing, di mana kita diminta untuk membangun Red-Black Tree dari sekumpulan bilangan masukan kemudian menjawab beberapa query berupa permintaan hasil traversal tertentu. Berbeda dengan soal-soal sebelumnya yang lebih kepada representasi graph dan eksplorasi jalur, di sini inti permasalahannya adalah bagaimana memanfaatkan sifat BST yang terjaga balanced-nya untuk menghasilkan urutan node sesuai jenis traversal yang diminta.

Kita bisa memanfaatkan method publik yang tersedia, yaitu `root()` dan `nil()` untuk mengakses struktur internal tree yang disediakan. Dua method ini memberikan kita pointer ke node root dan sentinel nil, yang kemudian digunakan sebagai titik masuk untuk menulis fungsi traversal rekursif.

Ide utama penyelesaian dimulai dengan membaca N bilangan dan memasukkannya satu per satu ke dalam tree. Namun ada aturan khusus: duplikat harus diabaikan. Karena method `insert()` pada class yang diberikan tidak secara eksplisit menangani duplikat, maka sebelum melakukan insert kita perlu memastikan bahwa nilai yang akan dimasukkan belum ada di dalam tree dengan memanggil `contains()`. Jika `contains()` mengembalikan false, barulah `insert()` dieksekusi. Mekanisme ini menjaga agar setiap key dalam tree bersifat unik, yang merupakan prasyarat agar hasil traversal InOrder selalu menghasilkan urutan terurut dari kecil ke besar. Tanpa penanganan duplikat ini, struktur BST bisa menjadi ambigu dan hasil traversal mungkin tidak konsisten.

Setelah tree terbentuk, program membaca Q query. Setiap query berupa string yang menentukan jenis traversal yang diminta. Karena traversal yang digunakan adalah rekursif, kita memerlukan tiga pasang fungsi helper yang bekerja dengan pola visit yang berbeda. Fungsi preorder mencetak node saat ini sebelum rekursi ke subtree kiri dan kanan, fungsi inorder menunda pencetakan hingga seluruh subtree kiri selesai diproses, dan fungsi postorder menunda hingga kedua subtree selesai. Semua fungsi ini menggunakan nil sebagai kondisi terminasi, bukan `nullptr`, karena implementasi RedBlackTree yang diberikan menggunakan sentinel node untuk menandakan akhir cabang. Penggunaan pengecekan `node->isNil` atau `node == nil` menjadi kritis agar traversal berhenti pada leaf tanpa menyebabkan infinite recursion.

Output diformat sedemikian rupa sesuai jenis query. Ketika query bernilai ALL, ketiga traversal dicetak secara berurutan. Seluruh proses ini mengandalkan fakta bahwa Red-Black Tree tetap mempertahankan sifat BST meskipun dilakukan rotasi dan recoloring selama

insert, sehingga urutan InOrder selalu sorted ascending dan urutan PreOrder serta PostOrder selalu konsisten dengan struktur tree yang terbentuk pada saat itu.

TAUTAN GIT

Repo lengkap : <https://github.com/biahlil/Algoritma-dan-Struktur-Data.git>
Linked List : <https://github.com/DSA26-ULM/module-6-graph-and-tree-biahlil.git>